

UNIVERSIDADE FEDERAL DE JUIZ DE FORA
CYGNO - Grupo UFJF

Atividade: Introdução ao projeto de IC

Introdução e Objetivos

Esta atividade foi projetada para ser uma introdução ao problema de análise de dados que estamos enfrentando.

O objetivo principal é realizar uma análise exploratória completa do nosso conjunto de dados simulados. Ao final, você terá:

- Familiaridade com a natureza dos nossos dados.
- Experiência prática com as principais bibliotecas do Python (Uproot, Numpy, Matplotlib, Scipy). Principalmente Uproot que serve para abrir arquivos em formato ROOT (muito utilizado pelo CYGNO).
- Uma compreensão visual das diferenças e semelhanças entre eventos de Recuo Eletrônico (ER) e Recuo Nuclear (NR) em diferentes energias.
- Um conjunto de resultados que servirá como base para suas primeiras contribuições ao projeto.

Ferramentas recomendadas: Google Colab ou VSCode com ambiente Python configurado.

Parte 1: Familiarização com os Dados de Imagem

Nesta primeira parte, vamos trabalhar diretamente com as imagens dos eventos de ER e NR simulados para entender sua estrutura visual.

Cada um dos arquivos a serem usados nas próximas tarefas contém 5 imagens.

Dica para parte de plotar as imagens: use o cmap 'gray' e controle os parâmetros 'vmin' e 'vmax'.

Carregar e Visualizar Eventos *Truth*

Os eventos *"truth"* são as simulações puras, sem o ruído da câmera. Usaremos a biblioteca `uproot` para acessá-los.

Passo a passo para abrir os arquivos de imagem:

```
# Primeiro, instale o uproot (ex: no Colab)
# !pip install uproot matplotlib numpy

import uproot
import numpy as np

# Dados do evento
energia = 30 # keV
tipo_evento = 'ER' # ou 'NR'

# Caminho para o arquivo (path)
caminho_do_arquivo =
↪  f'dados_atividade/{energia}keV/{tipo_evento}_truth_{energia}keV.root'

# Abrir o arquivo com uproot
arquivo = uproot.open(caminho_do_arquivo)

# Conteúdo do arquivo
print("\n--- Conteúdo do Arquivo (chaves) ---")
chaves_no_arquivo = arquivo.keys()
print(chaves_no_arquivo)

# Carregar uma das imagens do arquivo

# Alternativa 1
ev = 0 # número do evento

nome_imagem_evento = chaves_no_arquivo[ev].split(';')[0]

print(f"\nSelecionando a imagem do evento {ev}: '{nome_imagem_evento}'")

img = arquivo[nome_imagem_evento].values()
img = img.T
print(np.shape(img))

# Alternativa 2
```

```python
ev = 0 # número do evento

# Primeiro digito = 0 -> ER
# Primeiro digito = 1 -> NR
if tipo_evento == 'ER':
    run = energia          # 00030 pra ER
if tipo_evento == 'NR':
    run = 10000 + energia # 10030 pra NR

print(f"\nSelecionando a imagem do evento {ev}: 'pic_run{run:05d}_ev{ev}'")

img = arquivo["pic_run%05d_ev%d"%(run, ev)].values()
img = img.T
print(np.shape(img))
```

- **Tarefa:** Carregue os arquivos de imagens dos eventos de ER e NR para todas as energias disponíveis (3, 6, 10, 30 keV).
- **Tarefa:** Para cada nível de energia, plote lado a lado um exemplo de imagem de um evento ER e um evento NR.
- **Tarefa:** Use funções das bibliotecas (numpy/scipy) para aplicar transformações de rotação e translação em uma imagem de evento. Plote a imagem original e as imagens transformadas.

Carregar e Visualizar Imagem de Pedestal (Ruído)

O pedestal representa o ruído eletrônico real da câmera. O passo a passo para abrir os arquivos de imagem de pedestal é similar ao anterior:

```python
import uproot
import numpy as np

# Caminho para o arquivo (path)
caminho_do_arquivo = 'dados_atividade/pedestal.root'

# Abrir o arquivo com uproot
arquivo = uproot.open(caminho_do_arquivo)

# Conteúdo do arquivo
print("\n--- Conteúdo do Arquivo (chaves) ---")
chaves_no_arquivo = arquivo.keys()
```

```python
print(chaves_no_arquivo)

# Carregar uma das imagens do arquivo

# Alternativa 1
ev = 0 # número do evento
nome_imagem_evento = chaves_no_arquivo[ev].split(';')[0]

print(f"\nSelecionando a imagem do evento {ev}: '{nome_imagem_evento}'")

img = arquivo[nome_imagem_evento].values()
img = img.T
print(np.shape(img))

# Alternativa 2
ev = 0 # número do evento
run = 1

print(f"\nSelecionando a imagem do evento {ev}: 'pic_run{run:05d}_ev{ev}'")

img = arquivo["pic_run%05d_ev%d"%(run, ev)].values()
img = img.T
print(np.shape(img))
```

- **Tarefa:** Carregue o arquivo de imagens de pedestal (`pedestal.root`).
- **Tarefa:** Plote 2 ou 3 exemplos de imagens de pedestal.

Criando Imagens Realistas

Vamos juntar a simulação pura com o ruído real.

- **Tarefa:** Escolha uma imagem de evento *"truth"* e uma imagem de pedestal aleatória. Some as duas imagens.
- **Tarefa:** Plote as três imagens lado a lado: a imagem *"truth"*, a imagem do pedestal e a imagem final resultante.

Parte 2: Análise das Características Reconstruídas

As imagens realistas (truth + ruído) passam por um algoritmo (*reconstruction*). Objetivo desse algoritmo é encontrar na imagem o grupo de pixels (cluster) do evento (no

nosso caso o 'truth') em meio ao ruído da câmera. Ao detectar os clusters na imagem, o algoritmo extrai variáveis referentes aos cluster (ex: largura, comprimento, integral, etc).

A saída desse algoritmo é um arquivo .root (reco3D). Os arquivos a serem usados nas próximas tarefas são referentes a um total de 1000 imagens de ER e 1000 de NR para cada nível de energia.

Carregar os Dados Reconstruídos

O arquivo `reco3D` contém uma TTree (uma "tabela"dentro do arquivo ROOT) com cada variável e seus valores.

Passo a passo para abrir o arquivo `reco3D`:

```
# --- 1. Carregar o arquivo reco3D ---

# Dados do evento
energia = 30 # keV
tipo_evento = 'ER' # ou 'NR'

# Caminho para o arquivo (path)
caminho_do_arquivo =
↪  f'dados_atividade/{energia}keV/{tipo_evento}_reco3D_{energia}keV.root'

arquivo = uproot.open(caminho_do_arquivo)

# Acessa a TTree, que se chama "Events"
tree = arquivo['Events']

# --- 2. Visualizar todas as variáveis disponíveis ---
# O método .show() é o equivalente do uproot ao .Print() do ROOT para TTrees.
# Ele mostra um resumo claro de todos os branches (variáveis).
print("\n--- Estrutura da TTree 'Events' ---")
tree.show()
```

Passo a passo para selecionar variáveis da TTree:

```
# --- 3. Selecionar 4 features e obter os vetores ---
# Lista das variáveis (features) de interesse
features_selecionadas = [
    'sc_size',
    'sc_integral',
    'sc_length',
```

```
    'sc_width'
]

print(f"\n--- Carregando as features selecionadas: {features_selecionadas} ---")

# Extrair as varíaveis da tree
# O resultado é um dicionário numpy
dados = tree.arrays(features_selecionadas, library = 'np')

# Vamos inspecionar o resultado
print("Chaves no dicionário:", dados.keys())

# Exemplo: Printar os 5 primeiros valores da feature 'sc_size'
feature_exemplo = 'sc_size'
print(f"\nPrimeiros 5 valores de '{feature_exemplo}':")
# O [0] é porque 'sc_size' é um array de arrays (um por evento, podendo ter mais de um cluster)
# Pegamos o valor do primeiro cluster de cada um dos 5 primeiros eventos.
primeiros_valores = [arr[0] for arr in dados[feature_exemplo][:5] if len(arr) > 0]
print(primeiros_valores)
```

- **Tarefa:** Carregue o arquivo `reco3D` e visualize as variáveis disponíveis.
- **Tarefa:** Selecione algumas variáveis da TTree e inspecione o resultado, como foi feito no último passo a passo. Pode fazer prints diferentes para ir vendo como funciona.

Análise por Histogramas

No nosso grupo, fazemos muitas análises de dados através de histogramas.

- **Tarefa:** Para **cada nível de energia** (3, 6, 10, 30 keV):
  1. Selecione 4 variáveis do arquivo reco3D, e carregue os dicionários correspondentes de ER e NR daquela energia.

     Observação: algumas variáveis do reco3D não são referentes aos clusters encontrados, portanto selecione 4 variáveis da Tabela 1, no final do roteiro da atividade.
  2. Para **cada uma das variáveis selecionadas**, plote um histograma com a distribuição dos valores para eventos ER e NR sobrepostos no mesmo gráfico. Use cores diferentes e transparência (`alpha`).

Ajuste de Curva Gaussiana

- **Tarefa:** Inspecione os histogramas que você gerou, escolha um que pareça Gaussiano e ajuste uma função Gaussiana aos dados do histograma.

  Dica: cmos_rms.

  Sugestão: Use a função `scipy.optimize.curve_fit`.

- **Tarefa:** Plote o histograma original e, sobre ele, a curva Gaussiana com os parâmetros ótimos.

Parte 3: Apresentação dos Resultados

- **Tarefa:** Prepare uma breve apresentação de slides para mostrar os resultados encontrados.

Dúvidas?

Luan
email: luan.gomes@estudante.ufjf.br

Igor
email: igorpains@gmail.com

Tabela 1: Lista de Variáveis e Suas Descrições

Variável	**Descrição**
cmos_integral	Contagem integral de todo o sensor CMOS
cmos_mean	Média da contagem de todo o sensor CMOS
cmos_rms	RMS da contagem de todo o sensor CMOS
sc_size	Número de pixels do cluster (sem supressão de zeros)
sc_nhits	Número de pixels do cluster acima do limiar
sc_integral	Integral não calibrada da contagem de pixels no cluster
sc_corrintegral	Integral do cluster com correção de densidade
sc_rms	RMS da contagem de pixels no cluster
sc_theta	Ângulo polar de inclinação do eixo principal do cluster
sc_length	Comprimento do eixo principal do cluster
sc_width	Comprimento do eixo menor do cluster
sc_longrms	RMS truncado do cluster ao longo do eixo principal
sc_latrms	RMS truncado do cluster ao longo do eixo menor
sc_lfullrms	RMS completo do cluster ao longo do eixo principal
sc_tfullrms	RMS completo do cluster ao longo do eixo menor
sc_lp0amplitude	Amplitude do pico principal do perfil longitudinal
sc_lp0prominence	Proeminência do pico principal do perfil longitudinal
sc_lp0fwhm	Largura à meia altura do pico principal (longitudinal)
sc_lp0mean	Posição média do pico principal (longitudinal)
sc_tp0fwhm	Largura à meia altura do pico principal (transversal)
sc_tgaussamp	Amplitude do perfil gaussiano transversal
sc_tgaussmean	Posição média do perfil gaussiano transversal
sc_tgausssigma	Desvio padrão do perfil gaussiano transversal
sc_tchi2	Chi-quadrado do ajuste gaussiano (transversal)
sc_lgaussamp	Amplitude do perfil gaussiano longitudinal
sc_lgaussmean	Posição média do perfil gaussiano longitudinal
sc_lgausssigma	Desvio padrão do perfil gaussiano longitudinal
sc_lchi2	Chi-quadrado do ajuste gaussiano (longitudinal)